

```
# Show the new account
accountsDict[newAccountNumber].show()
```

Listing 4-5: Modified test code to store account numbers and objects in a dictionary

Running this code yields results almost identical to those of the previous examples. We start with an empty dictionary of accounts ❶, and initialize our `nextAccountNumber` variable to 0 ❷. Every time we instantiate a new account, we add a new entry into the dictionary of accounts using the current value of `nextAccountNumber` as a key and the `Account` object as the value ❸. We do this for each customer, as you can see for Mary ❺. Every time we create a new account, we increment `nextAccountNumber` to prepare for the next account ❹. With account numbers as keys in a dictionary, if a customer closes their account, we can eliminate that key and value from the dictionary without affecting any other accounts.

Building an Interactive Menu

With our `Account` class working correctly, we'll make the main code interactive by asking the user to tell us what operation they would like to do: get the balance, make a deposit, make a withdrawal, or open a new account. In response, our main code will gather the needed information from the user, starting with their account number, and call the appropriate method of the user's `Account` object.

As a shortcut, we will again prepopulate two accounts, one for Joe and one for Mary. [Listing 4-6](#) shows our expanded main code, which uses a dictionary to keep track of all the accounts. I've omitted the code that creates the accounts for Joe and Mary and adds those to the dictionary of accounts for brevity, as it's the same as in [Listing 4-5](#).

File: BankOOP4_InteractiveMenu/Main_Bank_Version4.py

```
# Interactive test program creating a dictionary of accounts
# Version 4, with an interactive menu

from Account import *
```